Sarabo — Presentation Script (long version, 12–14 minutes)

Same 10 slides and the same story as the 5-minute script, with room to explain. About 1,800 words. Each slide ends with a bridge line that walks you into the next one. Cues in *[square brackets]* are for you, not for the audience: *[pause]* means one breath, *[point]* means point at the slide.

Slide 1 · Title

Assalamu alaikum, and good morning to the respected board. I'm Jahid Hasan, ID 222-115-116, Batch 57, section C, from the Department of CSE at Metropolitan University, Sylhet. My project was supervised by Shrabanti Chowdhury, Senior Lecturer, and it's called **Sarabo, an electronics repair service management platform.**

The line on the slide says what I wanted it to do: *repair your device with confidence.* I built all of it myself, from the requirements to the server, the website, the tests and the deployment. And it isn't only on my laptop; it's running on the internet right now.

Bridge: So why build this at all? Let me tell you about someone.

Slide 2 · The problem

This is Rahim. *[pause]* He's a student in Ambarkhana, here in Sylhet. One evening his Samsung Galaxy A54 slips out of his hand. The screen cracks, a green line appears, and the touch stops working.

He goes to the nearest shop. He asks, "How much?" They say, "Around 2,500, we'll see after opening it." They keep the phone for a week. He calls three times just to ask where it is. When he finally collects it, the bill is 4,500. Nothing on paper explains why.

And ten days later the touch fails again. The shop says, "You dropped it again." Rahim can't prove anything. No quote, no receipt, no record of what the phone looked like when he handed it over.

Now look at the other side of the counter. *[point]* Karim is a good technician, but he gets work only when someone recommends him. He writes quotes on paper, and sometimes a customer changes their mind and simply doesn't pay. And whoever runs a repair service can't see which jobs are stuck, and handles refunds by hand.

So you have three people, and none of them trusts the other two. That's why so many repairs just never happen.

Bridge: So I asked myself: can one system make this fair and clear for all three of them?

Slide 3 · The solution

That's what Sarabo is. It turns a repair into **one tracked, priced workflow** for three roles.

The **customer** asks for a repair, compares offers, pays, and always knows where the device is. The **technician** finds jobs nearby, sends a quote, does the work and gets paid for sure. The **admin** approves technicians, sets prices, handles problems and reads the reports.

There are 45 services across 8 types of device, all priced in taka. Every request shows a simple progress bar, Request, Inspect, Approve, Repaired, with a "Your next step" box so nobody is ever confused about what to do.

One small thing I'm proud of: if no technician covers Rahim's area for that repair, Sarabo tells him straight away, instead of leaving him waiting for days. And it counts those requests, so the admin knows where to find more technicians.

Bridge: Let me walk you through Rahim's repair, step by step.

Slide 4 · How it works

[point] There are six steps.

One. Rahim types the problem the way he'd say it, "screen venge gese, touch kaj kore na", and AI fills the form for him. He also picks a two-hour pickup slot.

Two. Technicians nearby who actually repair phones see the job and send offers.

Three. Rahim chooses one and pays a small inspection fee. That fee is what books the technician.

Four. The technician collects the phone, writes down its condition with Rahim, inspects it, and sends an itemised quote: labour 800, parts 3,200.

Five. Rahim approves and pays only what's left, 3,800, because his 200 is already counted.

Six. He gets his phone back. Karim receives 90 percent, Sarabo keeps 10, and a 30-day warranty starts.

And I wanted it to be fair to both sides. If Karim doesn't show up, Rahim gets a full refund. If Rahim cancels at least two hours before pickup, also a full refund. And Karim's money is held until Rahim confirms he has his phone, so the customer is protected, and the technician knows the money is really there.

Bridge: To make all of this work, the data has to be shaped the right way.

Slide 5 · ER diagram

[point] This is the data model, stored in MongoDB.

A **user** can create many repair requests, and the same user can also become a **technician**. Each **repair request** is priced by one **service** from the catalogue. It receives many **offers**, and it has its **payments**, at most one **review**, and at most two **warranty claims**. The double bar means "one", the crow's foot means "many".

You'll notice the quote and the inspection aren't separate tables. They live inside the request, because they only ever belong to that one repair. That's also why I chose MongoDB: one repair, with its quote, history and payout, fits naturally into one document.

Two small design choices I'd like to mention. When a request is created, its price is copied into it, so if the admin changes prices later, Rahim's repair doesn't change. And a technician's wallet balance is never stored as a number; it's calculated from the records every time, so it can't quietly go wrong.

Bridge: Now let's see what Rahim actually sees on screen.

Slide 6 · Customer: ranked offers and the quote

When the offers come in, Rahim isn't left guessing which technician to trust. Sarabo ranks them using five things it already knows: rating, reliability, price, whether the technician is in his district, and how fast they usually reply. *[point]* The small tags, like "Best match" or "Same district", tell him *why* someone is on top.

Rating counts for 30 percent, reliability 25, price 20, district 15 and reply time 10. A brand-new technician gets a neutral score, so new people can still find work. And the final choice is always Rahim's.

Then, after inspection, *[point]* the quote is right there in writing. 4,000 taka. The 200 he already paid is taken off. He pays 3,800, in one step, and he can download a PDF receipt straight away.

Bridge: But the story doesn't end when Rahim pays.

Slide 7 · Warranty, and no stuck repairs

Remember Rahim's touch failing ten days later? *[pause]* On Sarabo, that's exactly what the warranty is for.

Every repair has **30 days of warranty**. Rahim opens a claim, the same technician comes back and checks it for free, and if it's covered, it's repaired again for free, or refunded. If the technician says "not covered", say new water damage, he has to give a reason, and Rahim can ask the admin to look at it. The condition record from the pickup is the evidence that protects both of them.

The second thing I really cared about is that **no request can get stuck**. Every waiting step has a deadline. An approved quote that isn't paid closes after three days. If Rahim forgets to confirm he got his phone, it's confirmed for him after three days. If a technician can't finish, Rahim is refunded. And anything that stalls shows up on the admin's screen.

Bridge: Some of these steps get a little help from AI.

Slide 8 · Applied AI

I used Google Gemini in four places. It turns Rahim's own words into a filled form. It helps Karim write a clear, polite note with his offer, in English or Bangla. It answers questions in the support chat. And it explains the admin's weekly numbers in plain sentences.

But I was careful here, because AI can be confidently wrong. So the rule is: **AI suggests, people decide.** AI never sets a price. It can only pick services that really exist, and the server checks every answer again. Phone numbers are removed from offer notes, so deals stay on the platform. If the weekly summary mentions a number that isn't in the real data, that sentence is dropped. And no names or personal details are ever sent.

If AI is slow or switched off, nothing breaks; you just fill the form yourself.

Bridge: None of this means anything unless the foundation underneath is solid.

Slide 9 · Architecture

[point] On the front there's a **React 19** website. It talks to an **Express 5** API running on Vercel in Mumbai, right next to the **MongoDB Atlas** database. Around them are Firebase for sign-in, Stripe for payments, Supabase for photos, Gemini for AI, and Gmail for emails.

A quick story about that Mumbai part. At first the API ran in the US while the database was in Mumbai, and the first request took more than ten seconds. Moving it next to the database brought that down to about three seconds, and normal requests from 0.75 to 0.2 seconds.

Because real money moves through this system, I followed three rules. *[pause]* First, **the server decides**: it checks who you are and what state the repair is in on every single action, and it never just trusts the browser. Second, **all or nothing**: booking, payment and refunds run inside database transactions, so nothing half-happens. Third, **exactly once**: a payment is confirmed only by Stripe's signed message, and even if it arrives twice, or two people click at the same moment, nobody pays twice and no job is booked twice.

Bridge: So how do I know all of this actually works?

Slide 10 · Results, limits and what's next

I tested it, a lot. There are **2,761 automated server tests** that make real requests against a real database, including tests where two admins act at the same moment. And there are **58 browser tests** on both a desktop and a phone screen. All of them pass, and they run automatically every time I push code. All **twelve objectives** I set at the start were met, and the system is live.

I also want to be honest about what it doesn't do yet. Stripe is in test mode, because it doesn't work for real in Bangladesh. Payouts to technicians through bKash are sent by the admin by hand. And it hasn't had real customers yet, or a load test.

If I continue, the next steps are a local payment gateway like SSLCOMMERZ or bKash, and a mobile app with a full Bangla interface, so someone like Rahim can do all of this from his phone, in his own language.

[pause] So, Rahim gets a fair, written price, he can follow his phone at every step, and he has a warranty. And Karim gets steady work and knows he'll be paid.

Thank you. Now let me show you Rahim's repair, live.

Making it sound like you

- **Learn the story, not the lines.** If you remember Rahim, the cracked screen, the 4,500 bill and the warranty, the rest follows naturally.
- **Slow down on the numbers.** Say these clearly: 45 services, 8 device types, 90 percent, 30 days, 2,761 and 58 tests.
- **Look at the board, not the slide.** Glance at the slide only when you *[point]*.
- **Use the bridges.** They are what make it sound like one talk instead of ten separate slides.
- **If you forget a line, skip it.** Nobody knows the script but you; just go to the next point.